

Instagram Feed Architecture Design Document

Production Spec, Complete Data Store Rationale, End-to-End Write & Read Paths, and Local Docker Compose Rig

AUTH: PRINCIPAL ENGINEER SPEC

LOCAL RUNTIME: DOCKER COMPOSE

HYBRID FAN-OUT

VECTOR DISCOVERY

1. Executive Summary & Core Constraints

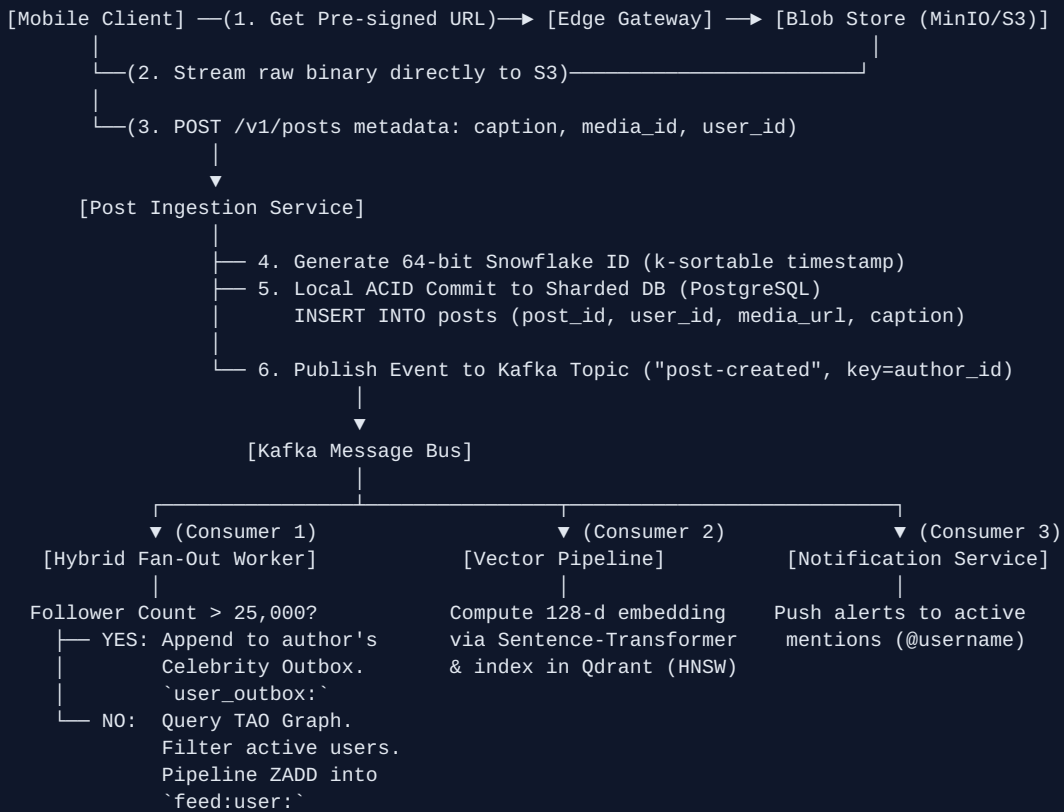
This document establishes the technical design for a hyperscale feed system serving 500M Daily Active Users (DAU) at ~80,000 read QPS with a strict p99 latency target under 200ms. The architecture rejects monolithic patterns in favor of an asymmetric hybrid fan-out model, decoupled event streams, in-memory timeline caches, and two-tower vector discovery. Crucially, the entire stateful infrastructure is reproducible locally using a unified **Docker Compose** environment for hands-on benchmarking, debugging, and interview demonstrations.

2. Comprehensive Data Store Selection & Evaluation

System Layer	Chosen Production Tech	Local Docker Equivalent	Architectural Rationale & Trade-Offs
Primary Relational Metadata	Sharded PostgreSQL (InnoDB / Vitess)	postgres:16-alpine	Horizontally sharded by <code>user_id</code> . Executes single-shard ACID transactions for posts, media metadata, and account state without distributed locking. Supports 64-bit Snowflake ordering.
Social Graph Engine	Meta TAO (Objects & Associations)	PostgreSQL <code>follows</code> table + Redis Cache	Graph edges (<code>follower_id</code> → <code>followee_id</code>) require bidirectional lookups. Production TAO provides a read-through graph cache; locally modeled via indexed B-Tree joins cached in Redis.
Feed Timeline Inboxes	RocksDB LSM on NVMe / Redis on Flash	redis:7.2-alpine	Maintains bounded sorted sets (<code>ZSET</code> , max 800 items) per active user. Redis Sorted Sets model time-based scoring (<code>ZREVRANGEBYSCORE</code>) in RAM with sub-2ms reads.
Out-of-Network Vector Recs	FAISS / ScaNN / Milvus Cluster	qdrant/qdrant:latest	Stores 128-d dense embeddings of post semantics. Hierarchical Navigable Small World (HNSW) indexing achieves Approximate Nearest Neighbor (ANN) searches in <10ms.
Decoupled Event Streaming	Apache Kafka Cluster	confluentinc/cp-kafka:latest	Decouples post creation from asynchronous fan-out workers, search indexing, and real-time counter updates. Partitioned by <code>author_id</code> for in-order delivery.
Object / Blob Storage	AWS S3 / Google Cloud Storage	minio/minio:latest	Direct-to-blob binary storage for high-res images and HLS video chunks via pre-signed URLs, preventing media bytes from choking application servers.

3. End-to-End Write Path (Media & Post Publishing)

The write path is optimized for immediate client acknowledgement, low connection contention, and decoupled fan-out processing:



1. **Direct Storage Streaming:** The client acquires a signed upload ticket and uploads chunked media directly to MinIO/S3, completely bypassing the core backend CPU and memory.
2. **Atomic DB Commit:** Metadata is committed into PostgreSQL on the shard owning `user_id`. The Snowflake ID guarantees global ordering without multi-node locks.
3. **Kafka Asynchronous Handoff:** Ingestion publishes a `PostCreatedEvent` and immediately returns HTTP 201 Created to the user within 50ms.
4. **Hybrid Fan-Out Logic:**
 - If author followers $\leq 25,000$: Worker batches Redis commands (`ZADD feed:user:<follower_id> <ts> <post_id>`) for users active within the last 7 days, capping each inbox at 800 items.
 - If author followers $> 25,000$: Worker appends the post ID solely to the author's singular outbox set (`celebrity:outbox:<author_id>`).

4. End-to-End Read Path (Aggregation, Ranking & Pagination)

The read path implements a pull-driven multi-stage candidate funnel to return 20 hyper-personalized posts in $<100\text{ms}$:



Composite Ranking Scoring Objective:

$$Score = (w_1 \times P(Like)) + (w_2 \times P(Comment)) + (w_3 \times P(Share)) + (w_4 \times P(Dwell > 5s)) - (w_5 \times P(Hide))$$

5. Local Docker Compose Infrastructure Specification

The complete production stack is declared below in standard Docker Compose format. It provisions primary metadata persistence, graph relations, timeline caching, vector indexing, distributed messaging, and object storage locally on a single machine:

version: '3.8'

services:

1. Primary Metadata Database & Social Graph (PostgreSQL)

postgres:

image: postgres:16-alpine
container_name: ig-postgres
environment:
 POSTGRES_USER: ig_admin
 POSTGRES_PASSWORD: ig_password
 POSTGRES_DB: instagram_core

ports:

- "5432:5432"

volumes:

- pgdata:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U ig_admin -d instagram_core"]
interval: 5s
timeout: 5s
retries: 5

2. Timeline Cache Tier (Redis)

redis:

image: redis:7.2-alpine
container_name: ig-feed-cache
command: redis-server --appendonly yes --maxmemory 512mb --maxmemory-policy allkeys-lru
ports:

- "6379:6379"

volumes:

- redisdata:/data

3. Vector Database for Out-of-Network Recommendations (Qdrant)

qdrant:

image: qdrant/qdrant:v1.8.4
container_name: ig-vector-engine

ports:

- "6333:6333" # HTTP REST API
- "6334:6334" # gRPC API

volumes:

- qdrantdata:/qdrant/storage

4. Asynchronous Message Broker (Apache Kafka via KRaft - No Zookeeper)

kafka:

image: confluentinc/cp-kafka:7.5.1
container_name: ig-kafka

ports:

- "9092:9092"

environment:

KAFKA_NODE_ID: 1
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: 'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT'
KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092'
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
KAFKA_PROCESS_ROLES: 'broker,controller'
KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:29093'
KAFKA_LISTENERS: 'PLAINTEXT://0.0.0.0:29092,CONTROLLER://0.0.0.0:29093,PLAINTEXT_HOST://0.0.0.0:9092'
KAFKA_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'
KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
KAFKA_LOG_DIRS: '/tmp/kraft-combined-logs'
CLUSTER_ID: 'MkU30EVbNTcwNTJENDM2Qk'

5. Local S3-Compatible Blob Storage (MinIO)

minio:

image: minio/minio:latest
container_name: ig-blob-storage
command: server /data --console-address ":9001"
environment:

MINIO_ROOT_USER: minioadmin
MINIO_ROOT_PASSWORD: minioadminpassword

ports:

- "9000:9000" # S3 API endpoint

```

- "9001:9001" # Web Admin Console
volumes:
- miniodata:/data

volumes:
pgdata:
redisdata:
qdrantdata:
miniodata:

```

6. PostgreSQL Schema Initialization (DDL)

Execute the following DDL script inside the `postgres` container to establish the relational tables and index paths:

```

-- Posts Table (Sharded logically by user_id)
CREATE TABLE IF NOT EXISTS posts (
  post_id BIGINT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  media_url VARCHAR(512) NOT NULL,
  media_type SMALLINT NOT NULL, -- 1: Image, 2: Video, 3: Carousel
  caption TEXT,
  like_count INT DEFAULT 0,
  comment_count INT DEFAULT 0,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_user_posts ON posts (user_id, created_at DESC);

-- Social Graph Edges (Follows)
CREATE TABLE IF NOT EXISTS follows (
  follower_id BIGINT NOT NULL,
  followee_id BIGINT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  PRIMARY KEY (follower_id, followee_id)
);
CREATE INDEX idx_followee_lookup ON follows (followee_id);

-- User Profiles & Celebrity Flagging
CREATE TABLE IF NOT EXISTS users (
  user_id BIGINT PRIMARY KEY,
  username VARCHAR(64) UNIQUE NOT NULL,
  follower_count INT DEFAULT 0,
  is_celebrity BOOLEAN GENERATED ALWAYS AS (follower_count > 25000) STORED,
  last_active_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

```

7. Principal Architectural Trade-Offs Matrix

Design Decision	Chosen Approach	Alternative Rejected	Core Engineering Justification
Fan-Out Routing	Dynamic Hybrid (25k cutoff)	Pure Push or Pure Pull	Pure push crashes worker queues on 100M-follower posts; pure pull saturates database connection pools under 80k read QPS. Hybrid eliminates both failure modes.
Timeline Persistence	Flash SSD / Redis Sorted Sets	Dynamic SQL JOIN queries	Sorting 800 followees across millions of rows at runtime is $O(F \cdot P \cdot \log P)$ and saturates disk I/O. Redis in-memory ZSETs provide $O(\log N)$ slicing in <2ms.
Pagination Protocol	Encrypted AES-256 Cursors	SQL <code>OFFSET / LIMIT</code>	Offset pagination degrades as depth increases ($O(N)$ index traversal) and suffers from pagination drift (duplicate posts on scroll). Cursors anchor deterministic sessions.
Recommendation Retrieval	Two-Tower HNSW Vector Index	Collaborative Filtering Matrix	Deep matrix factorization fails sub-10ms SLAs and struggles with cold-start items. HNSW vector search decouples item indexing from real-time user taste projection.